

**Sistem Pendukung Keputusan
(DSS) Rekomendasi Wisata Berbasis
Graph**



OLEH :

I KADEK DWI MANIK SUTEJA (2501010107)

DEWA GEDE BAGUS PUTRA PRAMANA (2501010109)

I NYOMAN DIRGA WIRYADINATA (2501010116)

**PRODI INFORMATIKA
Institut Bisnis dan Teknologi Indonesia
TAHUN 2025/2026**

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan proyek yang berjudul **“Sistem Pendukung Keputusan (DSS) Rekomendasi Wisata Berbasis Graph”** dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas mata kuliah sekaligus sebagai implementasi penerapan struktur data graph dan algoritma Dijkstra pada sistem pendukung keputusan dalam menentukan rekomendasi wisata terbaik.

Kami menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi materi maupun penyusunan. Oleh karena itu, kami mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang.

Kami berharap laporan ini dapat memberikan manfaat bagi pembaca serta menjadi referensi dalam pengembangan sistem pendukung keputusan berbasis graph.

Akhir kata, kami mengucapkan terima kasih kepada semua pihak yang telah membantu dalam penyusunan laporan ini.

Bali, Mei 2026

Kelompok

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.1 Rumusan Masalah	2
1.2 Tujuan.....	2
1.3 Manfaat	2
BAB 2 DASAR TEORI	4
2.1 Sistem Pendukung Keputusan (DSS).....	4
2.2 Struktur Data Graph	4
2.3 Weighted Graph	5
2.4 Algoritma Dijkstra.....	5
2.5 Streamlit	6
2.6 Folium.....	6
BAB 3 ANALISIS DAN PERANCANGAN	8
3.1 Analisis Masalah	8
3.2 Analisis Kebutuhan Sistem	8
3.3 Desain Graph.....	9
3.4 Flowchart Sistem	10
3.5 Use Case.....	10
3.6 Struktur Node dan Edge.....	10
BAB IV IMPLEMENTASI SISTEM	12
4.1 Implementasi Sistem	12
4.2 Struktur Data Graph	13
4.3 Implementasi Algoritma Dijkstra	14
4.5 Implementasi Antarmuka Pengguna	15
4.7 Implementasi Visualisasi Peta	16
4.8 Implementasi Perhitungan Estimasi Waktu	17
4.9 Implementasi Detail Perjalanan	17
BAB V PENGUJIAN DAN ANALISIS SISTEM	18

5.1 Tujuan Pengujian	18
5.2 Metode Pengujian	18
5.3 Hasil Pengujian Sistem	18
5.4 Analisis Hasil Pengujian.....	20
5.5 Analisis Kompleksitas Algoritma	20
5.7 Kekurangan Sistem.....	21
5.8 Kesimpulan Pengujian	21
BAB VI KESIMPULAN DAN SARAN	22
6.1 Kesimpulan	22
6.2 Saran	22
DAFTAR PUSTAKA	23

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Bali merupakan salah satu daerah wisata yang sangat terkenal di Indonesia bahkan hingga ke luar negeri. Banyak wisatawan datang ke Bali untuk menikmati berbagai tempat wisata seperti pantai, wisata alam, wisata budaya, dan tempat hiburan lainnya. Banyaknya pilihan lokasi wisata di Bali membuat wisatawan memiliki banyak tujuan yang ingin dikunjungi dalam satu perjalanan.

Namun, banyak wisatawan sering mengalami kesulitan dalam menentukan rute perjalanan yang paling dekat dan efisien. Hal ini terjadi karena lokasi wisata di Bali tersebar di berbagai daerah sehingga membutuhkan perencanaan perjalanan yang baik. Jika wisatawan salah memilih jalur perjalanan, maka waktu tempuh akan menjadi lebih lama dan biaya perjalanan juga dapat bertambah.

Selain itu, sebagian wisatawan masih menentukan rute perjalanan secara manual dengan melihat peta atau mencari informasi sendiri melalui internet. Cara tersebut kurang efektif karena wisatawan harus membandingkan banyak lokasi dan memperkirakan sendiri jarak antar tempat wisata. Oleh karena itu, dibutuhkan sebuah sistem yang dapat membantu pengguna dalam menentukan jalur wisata terbaik secara otomatis.

Sistem Pendukung Keputusan atau Decision Support System (DSS) merupakan sistem yang dapat membantu pengguna dalam mengambil keputusan berdasarkan data dan proses perhitungan tertentu. Dalam penelitian ini, DSS digunakan untuk membantu wisatawan menentukan jalur perjalanan wisata yang lebih efektif dan efisien berdasarkan jarak terpendek.

Untuk membangun sistem tersebut digunakan struktur data graph. Graph digunakan karena mampu menggambarkan hubungan antar lokasi wisata. Setiap tempat wisata direpresentasikan sebagai node atau titik, sedangkan hubungan antar lokasi direpresentasikan sebagai edge atau garis yang memiliki bobot berupa jarak tempuh.

Algoritma yang digunakan dalam sistem ini adalah Algoritma Dijkstra. Algoritma Dijkstra digunakan untuk mencari jalur terpendek dari satu lokasi ke lokasi lainnya. Algoritma ini

bekerja dengan membandingkan total jarak dari setiap jalur yang tersedia hingga ditemukan rute dengan jarak paling kecil.

Pada sistem ini digunakan bahasa pemrograman Python dengan framework Streamlit untuk membuat tampilan aplikasi berbasis web. Selain itu, digunakan library Folium untuk menampilkan peta interaktif sehingga pengguna dapat melihat lokasi wisata dan rute perjalanan secara langsung pada peta.

Melalui sistem DSS Navigasi Wisata Bali ini, pengguna dapat memilih lokasi awal dan tujuan wisata, kemudian sistem akan menampilkan jalur tercepat beserta total jarak dan estimasi waktu perjalanan. Dengan adanya sistem ini diharapkan wisatawan dapat lebih mudah menentukan rute perjalanan wisata di Bali sehingga perjalanan menjadi lebih nyaman, cepat, dan efisien.

Selain membantu wisatawan, sistem ini juga dapat menjadi sarana pembelajaran mengenai penerapan struktur data graph, algoritma Dijkstra, serta pengembangan aplikasi berbasis Python dan Streamlit.

1.1 Rumusan Masalah

1. Bagaimana merancang DSS navigasi wisata Bali berbasis graph?
2. Bagaimana mengimplementasikan algoritma Dijkstra pada sistem?
3. Bagaimana menampilkan rute wisata pada peta interaktif?
4. Bagaimana sistem memberikan jalur terpendek antar lokasi wisata?

1.2 Tujuan

1. Merancang DSS navigasi wisata Bali.
2. Mengimplementasikan algoritma Dijkstra.
3. Menampilkan rute wisata pada peta digital.
4. Membantu wisatawan menentukan jalur tercepat.

1.3 Manfaat

Pembuatan Sistem Pendukung Keputusan (DSS) Navigasi Wisata Bali ini diharapkan dapat memberikan manfaat bagi pengguna maupun pengembang sistem. Bagi wisatawan, sistem ini dapat membantu dalam menentukan rute perjalanan yang lebih efektif dan efisien ketika mengunjungi berbagai tempat wisata di Bali. Dengan adanya informasi mengenai jalur terpendek, pengguna dapat menghemat waktu perjalanan serta mempermudah proses perencanaan wisata.

Selain itu, sistem ini juga memberikan kemudahan dalam memahami hubungan antar lokasi wisata melalui tampilan peta interaktif yang disediakan. Pengguna dapat melihat secara langsung jalur yang direkomendasikan oleh sistem sehingga perjalanan menjadi lebih terarah dan nyaman.

Bagi pengembang, pembuatan sistem ini dapat menjadi sarana untuk menerapkan ilmu yang telah dipelajari selama perkuliahan, khususnya pada mata kuliah Struktur Data, Algoritma, dan Sistem Pendukung Keputusan. Melalui proyek ini, penulis dapat memahami bagaimana struktur data graph digunakan untuk merepresentasikan hubungan antar lokasi serta bagaimana algoritma Dijkstra bekerja dalam menentukan jalur terpendek.

Selain itu, pengembangan sistem ini juga memberikan pengalaman dalam membangun aplikasi berbasis web menggunakan Python, Streamlit, dan Folium. Pengalaman tersebut dapat menjadi bekal dalam mengembangkan aplikasi yang lebih kompleks di masa mendatang.

Dari sisi akademik, penelitian ini diharapkan dapat menjadi referensi bagi mahasiswa lain yang ingin mempelajari atau mengembangkan sistem pendukung keputusan berbasis graph. Hasil penelitian ini juga dapat dijadikan sebagai contoh penerapan teori yang dipelajari di bangku perkuliahan ke dalam sebuah aplikasi yang dapat digunakan untuk menyelesaikan permasalahan nyata.

Secara umum, sistem yang dibangun diharapkan mampu memberikan solusi dalam membantu wisatawan menentukan rute perjalanan wisata yang lebih optimal sekaligus menjadi media pembelajaran mengenai penerapan algoritma graph dalam bidang pariwisata

BAB 2

DASAR TEORI

2.1 Sistem Pendukung Keputusan (DSS)

Decision Support System (DSS) merupakan sistem berbasis komputer yang digunakan untuk membantu proses pengambilan keputusan dengan memanfaatkan data, model, dan metode tertentu. DSS tidak menggantikan peran manusia dalam mengambil keputusan, tetapi berfungsi sebagai alat bantu yang menyediakan informasi dan rekomendasi yang relevan.

Karakteristik DSS:

- Mendukung proses pengambilan keputusan.
- Mengolah data menjadi informasi yang berguna.
- Memberikan alternatif solusi berdasarkan analisis data.
- Bersifat interaktif dan mudah digunakan.
- Membantu menyelesaikan masalah semi-terstruktur maupun tidak terstruktur.

Komponen utama DSS:

1. Database, yaitu tempat penyimpanan data yang digunakan oleh sistem.
2. Model Base, yaitu kumpulan model atau algoritma yang digunakan untuk melakukan analisis.
3. User Interface, yaitu antarmuka yang memungkinkan pengguna berinteraksi dengan sistem.

Pada penelitian ini, DSS digunakan untuk membantu wisatawan menentukan jalur perjalanan terbaik berdasarkan jarak terpendek antar lokasi wisata.

2.2 Struktur Data Graph

Graph merupakan struktur data yang digunakan untuk merepresentasikan hubungan antar objek. Graph terdiri dari kumpulan simpul (vertex) dan sisi (edge) yang menghubungkan simpul-simpul tersebut.

Secara matematis graph dituliskan sebagai:

- $G = (V, E)$

Keterangan:

- V (Vertex) merupakan kumpulan simpul.
- E (Edge) merupakan kumpulan sisi yang menghubungkan simpul.

Dalam sistem ini:

- Vertex digunakan untuk merepresentasikan lokasi wisata.
- Edge digunakan untuk merepresentasikan hubungan antar lokasi wisata.
- Bobot pada edge menunjukkan jarak antar lokasi.

Penggunaan graph sangat sesuai untuk sistem navigasi karena mampu menggambarkan jaringan jalan dan hubungan antar lokasi secara efektif.

2.3 Weighted Graph

Weighted graph adalah graph yang setiap sisinya memiliki nilai bobot tertentu. Bobot tersebut dapat berupa jarak, biaya, waktu tempuh, atau parameter lainnya. Pada sistem navigasi wisata Bali ini, bobot yang digunakan adalah jarak antar lokasi dalam satuan kilometer. Dengan adanya bobot tersebut, sistem dapat menghitung total jarak perjalanan dan menentukan jalur dengan jarak paling pendek.

Keuntungan penggunaan weighted graph antara lain:

- Memungkinkan perhitungan jalur terpendek.
- Mempermudah analisis hubungan antar lokasi.
- Cocok digunakan pada sistem navigasi dan transportasi.

2.4 Algoritma Dijkstra

Algoritma Dijkstra merupakan algoritma pencarian jalur terpendek yang diperkenalkan oleh Edsger W. Dijkstra pada tahun 1956. Algoritma ini digunakan untuk mencari jalur dengan total bobot terkecil dari satu titik awal menuju titik tujuan pada graph berbobot.

Langkah kerja algoritma:

1. Menentukan node awal.
2. Memberikan nilai jarak awal sebesar 0 pada node awal dan tak hingga pada node lainnya.
3. Memilih node dengan jarak terkecil yang belum dikunjungi.
4. Menghitung kemungkinan jarak baru ke node tetangga.
5. Memperbarui nilai jarak jika ditemukan jalur yang lebih pendek.
6. Mengulangi proses hingga seluruh node selesai diproses atau tujuan ditemukan.

Kelebihan algoritma Dijkstra:

- Akurat dalam menentukan jalur terpendek.
- Efisien untuk graph berbobot positif.
- Banyak digunakan pada sistem navigasi dan pemetaan.

2.5 Streamlit

Streamlit merupakan framework Python yang digunakan untuk membangun aplikasi web interaktif dengan cepat dan mudah. Framework ini memungkinkan pengembang membuat antarmuka pengguna tanpa harus menggunakan HTML, CSS, atau JavaScript secara langsung.

Pada sistem ini, Streamlit digunakan untuk:

- Menampilkan halaman utama aplikasi.
- Menyediakan menu pemilihan lokasi awal dan tujuan.
- Menampilkan hasil pencarian rute.
- Menampilkan informasi jarak dan estimasi waktu perjalanan.

2.6 Folium

Folium adalah library Python yang digunakan untuk membuat peta interaktif berbasis Leaflet.js. Library ini memungkinkan pengembang menampilkan data geografis secara visual dan interaktif.

Pada sistem ini, Folium digunakan untuk:

- Menampilkan peta wilayah Bali.
- Menampilkan marker lokasi wisata.
- Menampilkan jalur antar lokasi.
- Menampilkan rute terpendek hasil perhitungan algoritma Dijkstra.
- Menyediakan fitur zoom dan navigasi peta.

BAB 3

ANALISIS DAN PERANCANGAN

3.1 Analisis Masalah

Permasalahan utama yang dihadapi wisatawan adalah sulitnya menentukan jalur perjalanan yang paling efisien ketika mengunjungi berbagai lokasi wisata di Bali. Banyaknya pilihan destinasi dan jarak antar lokasi yang berbeda-beda menyebabkan wisatawan membutuhkan bantuan sistem yang mampu memberikan rekomendasi rute terbaik.

Untuk mengatasi permasalahan tersebut, dibangun sebuah DSS berbasis graph yang mampu menghitung jalur terpendek menggunakan Algoritma Dijkstra dan menampilkan hasilnya dalam bentuk peta interaktif.

3.2 Analisis Kebutuhan Sistem

Kebutuhan Fungsional

1. Sistem dapat menerima input lokasi awal dan tujuan.
2. Sistem dapat menghitung jalur terpendek menggunakan Algoritma Dijkstra.
3. Sistem dapat menampilkan total jarak perjalanan.
4. Sistem dapat menampilkan estimasi waktu perjalanan.
5. Sistem dapat menampilkan peta interaktif beserta rute perjalanan.
6. Sistem dapat menampilkan detail perjalanan antar lokasi.

Kebutuhan Non-Fungsional

1. Sistem memiliki tampilan yang mudah digunakan.
2. Sistem mampu memberikan hasil perhitungan dengan cepat.
3. Sistem dapat berjalan pada browser modern.
4. Sistem memiliki visualisasi yang informatif dan menarik.
5. Sistem mudah dikembangkan untuk penambahan lokasi wisata baru.

3.3 Desain Graph

Node yang digunakan dalam sistem terdiri dari berbagai lokasi wisata dan wilayah administratif di Bali seperti:

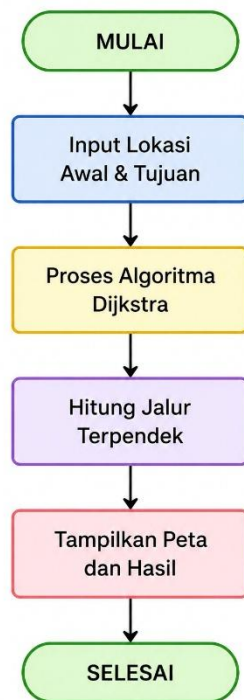
- Kuta
- Sanur
- Ubud
- GWK
- Tanah Lot
- Denpasar
- Gianyar Regency
- Badung Regency

Contoh hubungan antar node:

- Kuta $\rightarrow$ Sanur = 10 km
- Sanur $\rightarrow$ Ubud = 20 km
- Ubud $\rightarrow$ GWK = 40 km
- Kuta $\rightarrow$ Denpasar = 6 km

Hubungan tersebut digunakan sebagai dasar perhitungan jalur terpendek.

3.4 Flowchart Sistem



3.5 Use Case

Aktor

1. User (Wisatawan)

Aktivitas User

- Memilih lokasi awal perjalanan.
- Memilih lokasi tujuan.
- Menjalankan proses pencarian rute.
- Melihat hasil jalur terpendek.
- Melihat visualisasi peta.
- Melihat detail jarak dan estimasi waktu perjalanan.

3.6 Struktur Node dan Edge

Struktur Node

Field	Type
Nama Lokasi	String

Latitude	Float
Longitude	Float

Struktur Edge

Field	Tipe
Asal	String
Tujuan	String
Bobot	Integer

BAB IV

IMPLEMENTASI SISTEM

4.1 Implementasi Sistem

Pada penelitian ini dibangun sebuah Sistem Pendukung Keputusan (Decision Support System/DSS) Navigasi Wisata Bali yang bertujuan untuk membantu pengguna dalam menentukan rute perjalanan terpendek menuju lokasi wisata yang diinginkan. Sistem dikembangkan menggunakan bahasa pemrograman Python dengan memanfaatkan framework Streamlit sebagai antarmuka pengguna (user interface) dan Folium sebagai media visualisasi peta interaktif.

Metode yang digunakan dalam sistem adalah Algoritma Dijkstra yang diterapkan pada struktur data Graph berbobot (Weighted Graph). Setiap lokasi wisata direpresentasikan sebagai simpul (node), sedangkan hubungan antar lokasi direpresentasikan sebagai sisi (edge) yang memiliki bobot berupa jarak tempuh dalam satuan kilometer.

Secara umum, proses kerja sistem terdiri atas beberapa tahapan yaitu:

1. Pengguna memilih lokasi awal perjalanan.
2. Pengguna memilih lokasi tujuan perjalanan.
3. Sistem membaca data graph yang berisi lokasi dan jarak antar lokasi.
4. Algoritma Dijkstra melakukan perhitungan jalur terpendek.
5. Sistem menampilkan hasil rute terbaik.
6. Sistem menghitung total jarak perjalanan.
7. Sistem menampilkan estimasi waktu tempuh.
8. Sistem memvisualisasikan rute pada peta interaktif.

Dengan adanya sistem ini, pengguna dapat memperoleh rekomendasi rute yang optimal sehingga perjalanan menjadi lebih efisien.

4.2 Struktur Data Graph

Struktur data utama yang digunakan dalam sistem adalah Graph berbobot (Weighted Graph). Struktur graph dipilih karena mampu merepresentasikan hubungan antar lokasi secara efektif.

Dalam graph yang digunakan:

- Node (simpul) merepresentasikan lokasi wisata, kabupaten, dan desa.
- Edge (sisi) merepresentasikan jalur yang menghubungkan dua lokasi.
- Weight (bobot) merepresentasikan jarak antar lokasi dalam kilometer.

Implementasi graph disimpan menggunakan metode Adjacency List karena lebih efisien dalam penggunaan memori dibandingkan Adjacency Matrix.

Contoh representasi graph

```
107 graph = {
108   "Kuta": {"Sanur": 10, "Ubud": 35, "Denpasar": 6, "Badung Regency": 5, "Legian": 3, "Seminyak": 4, "Tuban": 3, "Kedonganan": 4},
109   "Sanur": {"Kuta": 10, "Ubud": 20, "Tanah Lot": 30, "Denpasar": 4, "Gianyar Regency": 25},
110   "Ubud": {"Kuta": 35, "Sanur": 20, "GWK": 40, "Gianyar Regency": 5, "Klungkung Regency": 35, "Karangasem Regency": 90},
111   "Tanah Lot": {"Sanur": 30, "GWK": 25, "Tabanan Regency": 15, "Buleleng Regency": 45},
112   "GWK": {"Ubud": 40, "Tanah Lot": 25},
113   "Denpasar": {"Kuta": 6, "Sanur": 4, "Badung Regency": 8},
114   "Badung Regency": {"Kuta": 5, "Denpasar": 8, "Tabanan Regency": 15, "Abiansemal": 10, "Mengwi": 10, "Petang": 12},
115   "Tabanan Regency": {"Badung Regency": 15, "Tanah Lot": 15, "Jembrana Regency": 60},
116   "Jembrana Regency": {"Tabanan Regency": 60},
117   "Buleleng Regency": {"Tanah Lot": 45},
118   "Bangli Regency": {"Gianyar Regency": 10, "Ubud": 30},
119   "Karangasem Regency": {"Ubud": 90},
120   "Klungkung Regency": {"Ubud": 35},
121   "Gianyar Regency": {"Sanur": 25, "Ubud": 5, "Bangli Regency": 10},
122   "Legian": {"Kuta": 3},
123   "Seminyak": {"Kuta": 4},
124   "Tuban": {"Kuta": 3},
125   "Kedonganan": {"Kuta": 4},
126   "Abiansemal": {"Badung Regency": 10},
127   "Mengwi": {"Badung Regency": 10, "Sempidi": 6},
128   "Sempidi": {"Mengwi": 6},
129   "Petang": {"Badung Regency": 12, "Carangsari": 8, "Belok Sidan": 9},
130   "Carangsari": {"Petang": 8},
131   "Belok Sidan": {"Petang": 9}
132 }
```

Berdasarkan data yang terdapat pada program, sistem memiliki 24 node yang saling terhubung dan membentuk jaringan rute wisata di Pulau Bali.

Keuntungan penggunaan Adjacency List antara lain:

1. Penggunaan memori lebih efisien.
2. Mudah dalam penambahan lokasi baru.
3. Cocok digunakan pada graph yang tidak terlalu padat.
4. Mempercepat proses pencarian node yang bertetangga.

4.3 Implementasi Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk menentukan jalur terpendek dari lokasi awal menuju lokasi tujuan. Algoritma ini bekerja dengan prinsip greedy, yaitu memilih jalur dengan biaya terkecil pada setiap langkah hingga mencapai tujuan.

Fungsi utama algoritma pada sistem dituliskan sebagai berikut:

```
def dijkstra(graph, start, end, stop_penalty=0):
```

Parameter yang digunakan:

Parameter	Keterangan
graph	Data graph lokasi
start	Lokasi awal
end	Lokasi tujuan
stop_penalty	Penalti jumlah pemberhentian

Langkah Kerja Algoritma

1. Inisialisasi

Node awal diberikan nilai jarak 0, sedangkan node lainnya diberikan nilai tak hingga (∞).

2. Pemilihan Node

Algoritma memilih node yang memiliki jarak terkecil dan belum dikunjungi.

3. Relaksasi Edge

Jarak ke seluruh tetangga dihitung menggunakan rumus:

$$\text{Jarak Baru} = \text{Jarak Saat Ini} + \text{Bobot Edge}$$

Jika jarak baru lebih kecil dibandingkan jarak sebelumnya, maka nilai jarak diperbarui.

4. Pengulangan

Proses dilakukan secara berulang hingga node tujuan ditemukan.

5. Pembentukan Jalur

Setelah node tujuan ditemukan, sistem melakukan proses pelacakan kembali (backtracking) untuk memperoleh jalur yang dilalui.

4.4 Implementasi Priority Queue

Untuk meningkatkan efisiensi perhitungan, sistem menggunakan struktur data Priority Queue melalui library HeapQ. Priority Queue digunakan untuk menyimpan node-node yang akan diproses berdasarkan prioritas jarak terkecil.

Implementasi:

```
</> Python
```

```
pq = [(0.0, 0.0, 0, start, [])]
```

Keuntungan penggunaan Priority Queue adalah:

1. Mempercepat pencarian node dengan jarak minimum.
2. Mengurangi kompleksitas waktu algoritma.
3. Cocok digunakan pada graph dengan jumlah node yang besar.

4.5 Implementasi Antarmuka Pengguna

Untuk meningkatkan efisiensi algoritma, sistem menggunakan struktur data Priority Queue yang diimplementasikan melalui library HeapQ.

Priority Queue digunakan untuk menyimpan node berdasarkan prioritas jarak terkecil.

Keuntungan penggunaan Priority Queue adalah:

1. Mempercepat proses pencarian node minimum.
2. Mengurangi jumlah proses pencarian.
3. Mengoptimalkan kinerja algoritma pada graph yang lebih besar.

Dengan adanya Priority Queue, algoritma tidak perlu melakukan pencarian secara berulang terhadap seluruh node.

4.6 Implementasi Antarmuka Pengguna

Antarmuka sistem dikembangkan menggunakan framework Streamlit.

Penggunaan Streamlit dipilih karena:

1. Mudah diintegrasikan dengan Python.
2. Mendukung visualisasi data secara interaktif.

3. Memiliki tampilan yang sederhana dan responsif.

Pada halaman utama tersedia beberapa komponen yaitu:

Sidebar

Sidebar digunakan untuk menampilkan seluruh kontrol sistem seperti:

- Pemilihan lokasi awal.
- Pemilihan lokasi tujuan.
- Pengaturan zoom peta.
- Preferensi jumlah pemberhentian.

Halaman Utama

Halaman utama digunakan untuk menampilkan:

- Peta interaktif.
- Hasil perhitungan.
- Informasi jarak.
- Estimasi waktu tempuh.

4.7 Implementasi Visualisasi Peta

Visualisasi peta merupakan salah satu fitur utama yang terdapat dalam sistem.

Peta dibangun menggunakan Folium yang memanfaatkan data OpenStreetMap.

Fitur visualisasi yang tersedia meliputi:

1. Marker lokasi wisata.
2. Marker wilayah administratif.
3. Jalur hasil perhitungan Dijkstra.
4. Layer Control.
5. MiniMap.
6. Fullscreen View.
7. Marker Cluster.

Visualisasi peta memudahkan pengguna untuk memahami rute yang direkomendasikan oleh sistem.

4.8 Implementasi Perhitungan Estimasi Waktu

Selain menghitung jarak, sistem juga memberikan estimasi waktu tempuh. Estimasi waktu dihitung berdasarkan total jarak yang diperoleh dari hasil pencarian jalur terpendek.

Sebagai contoh:

Total jarak perjalanan = 53 km

Kecepatan rata-rata kendaraan = 35 km/jam

Maka:

Waktu tempuh = $53 / 35$

= 1,51 jam

$\approx$ 91 menit

Informasi ini membantu pengguna dalam memperkirakan durasi perjalanan sebelum berangkat.

4.9 Implementasi Detail Perjalanan

Sistem menampilkan rincian perjalanan pada setiap segmen rute.

Contoh:

Mengwi $\rightarrow$ Ubud = 18 km

Ubud $\rightarrow$ Klungkung Regency = 35 km

Dengan adanya informasi tersebut, pengguna dapat mengetahui jarak antar lokasi yang akan dilalui selama perjalanan.

BAB V

PENGUJIAN DAN ANALISIS SISTEM

5.1 Tujuan Pengujian

Pengujian dilakukan untuk memastikan bahwa sistem telah berjalan sesuai dengan kebutuhan yang dirancang.

Tujuan pengujian meliputi:

1. Memastikan sistem dapat berjalan tanpa kesalahan.
2. Memastikan Algoritma Dijkstra menghasilkan jalur terpendek.
3. Memastikan perhitungan jarak sesuai dengan data graph.
4. Memastikan visualisasi peta dapat ditampilkan dengan benar.
5. Memastikan seluruh fitur dapat digunakan oleh pengguna.

5.2 Metode Pengujian

Metode yang digunakan adalah Black Box Testing.

Pengujian dilakukan dengan memberikan input tertentu kemudian mengamati output yang dihasilkan oleh sistem.

Fokus pengujian meliputi:

- Kesesuaian hasil perhitungan.
- Kebenaran rute.
- Fungsi antarmuka pengguna.
- Ketepatan visualisasi peta.

5.3 Hasil Pengujian Sistem

Pengujian Rute Mengwi ke Klungkung Regency

Input:

Lokasi Awal = Mengwi

Lokasi Tujuan = Klungkung Regency

Output:

Mengwi → Ubud → Klungkung Regency

Perhitungan:

Mengwi → Ubud = 18 km

Ubud → Klungkung Regency = 35 km

Total Jarak = 53 km

Status = Berhasil

Pengujian Rute Kuta ke Sanur

Input:

Kuta → Sanur

Output:

Kuta → Sanur

Total Jarak = 10 km

Status = Berhasil

Pengujian Rute Denpasar ke Jembrana Regency

Input:

Denpasar → Jembrana Regency

Output:

Denpasar → Badung Regency → Tabanan Regency → Jembrana Regency

Total Jarak = 83 km

Status = Berhasil

5.4 Analisis Hasil Pengujian

Berdasarkan seluruh pengujian yang dilakukan, sistem mampu menghasilkan jalur dengan total bobot terkecil. Hasil tersebut menunjukkan bahwa implementasi Algoritma Dijkstra telah berjalan sesuai teori shortest path problem.

Sebagai contoh, terdapat dua kemungkinan jalur dari Mengwi menuju Klungkung Regency.

- Alternatif pertama memiliki total jarak 53 km.
- Alternatif kedua memiliki total jarak 77 km.
- Karena 53 km lebih kecil daripada 77 km maka algoritma memilih alternatif pertama sebagai jalur terbaik.

Hal ini membuktikan bahwa sistem mampu memberikan solusi optimal berdasarkan data yang tersedia.

5.5 Analisis Kompleksitas Algoritma

Kompleksitas waktu Algoritma Dijkstra dengan Priority Queue adalah:

$$O((V + E) \log V)$$

Keterangan:

V = jumlah node

E = jumlah edge

Pada sistem yang dibangun terdapat 24 node dan sekitar 30 edge.

Dengan jumlah data tersebut, waktu komputasi yang dibutuhkan relatif kecil sehingga sistem mampu memberikan hasil secara cepat.

5.6 Kelebihan Sistem

1. Mampu menemukan jalur terpendek secara otomatis.
2. Menggunakan Algoritma Dijkstra yang optimal untuk graph berbobot positif.
3. Menampilkan visualisasi peta secara interaktif.

4. Menampilkan detail perjalanan secara lengkap.
5. Menampilkan total jarak dan estimasi waktu tempuh.
6. Memiliki antarmuka yang mudah digunakan.

5.7 Kekurangan Sistem

1. Data jarak masih bersifat statis.
2. Belum mempertimbangkan kondisi lalu lintas secara real-time.
3. Belum terintegrasi dengan Google Maps API.
4. Belum menggunakan data GPS secara langsung.
5. Jumlah lokasi wisata masih terbatas pada data yang dimasukkan ke dalam graph

5.8 Kesimpulan Pengujian

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, sistem berhasil menerapkan struktur data Graph dan Algoritma Dijkstra untuk menyelesaikan permasalahan pencarian rute terpendek pada lokasi wisata di Bali. Sistem mampu memberikan rekomendasi jalur yang optimal, menampilkan visualisasi peta secara interaktif, menghitung total jarak perjalanan, serta memberikan estimasi waktu tempuh. Dengan demikian sistem telah memenuhi tujuan penelitian dan dapat digunakan sebagai media pendukung pengambilan keputusan dalam perencanaan perjalanan wisata.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem pendukung keputusan rekomendasi rute wisata Bali berhasil dibangun menggunakan struktur data Graph dan Algoritma Dijkstra.

Sistem mampu menentukan jalur terpendek antar lokasi wisata berdasarkan bobot jarak yang tersedia pada graph. Hasil pengujian menunjukkan bahwa Algoritma Dijkstra dapat memberikan rekomendasi rute yang optimal dengan waktu komputasi yang cepat.

Selain itu, sistem mampu menampilkan visualisasi peta interaktif, total jarak perjalanan, estimasi waktu tempuh, dan detail perjalanan sehingga memudahkan pengguna dalam merencanakan perjalanan wisata.

6.2 Saran

Untuk pengembangan selanjutnya, beberapa saran yang dapat dilakukan adalah:

1. Menambahkan lebih banyak lokasi wisata.
2. Mengintegrasikan Google Maps API.
3. Menambahkan data lalu lintas real-time.
4. Menggunakan GPS untuk menentukan lokasi pengguna secara otomatis.
5. Menambahkan fitur rekomendasi objek wisata berdasarkan kategori dan rating pengguna.
6. Mengembangkan sistem menjadi aplikasi mobile berbasis Android atau iOS.

DAFTAR PUSTAKA

- Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1), 269–271.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. John Wiley & Sons.
- West, D. B. (2001). *Introduction to Graph Theory* (2nd ed.). Prentice Hall.
- Python Software Foundation. (2025). *heapq* — Heap queue algorithm. Diakses dari: <https://docs.python.org/3/library/heapq.html>
- Streamlit Inc. (2025). *Streamlit Documentation*. Diakses dari: <https://docs.streamlit.io>
- Folium Contributors. (2025). *Folium Documentation*. Diakses dari: <https://python-visualization.github.io/folium/latest/>
- Branca Contributors. (2025). *Branca Documentation*. Diakses dari: <https://python-visualization.github.io/branca/>
- Python Software Foundation. (2025). *Python Documentation*. Diakses dari: <https://www.python.org/doc/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Rosen, K. H. (2019). *Discrete Mathematics and Its Applications* (8th ed.). McGraw-Hill Education.